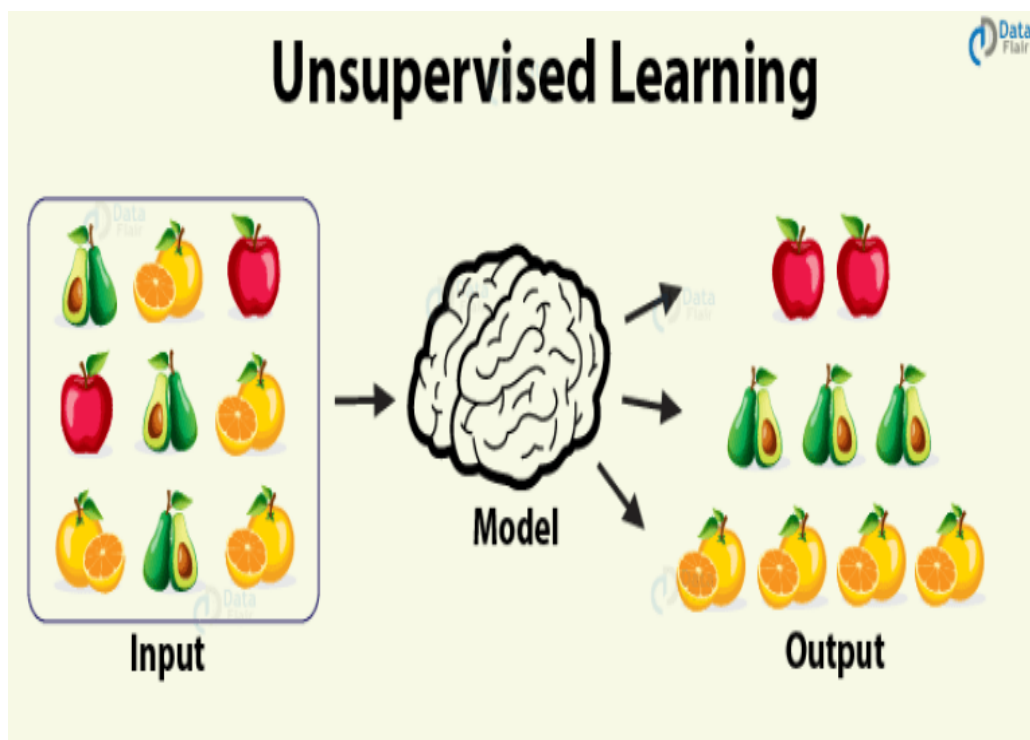


Unsupervised Learning Algorithms

What is unsupervised learning algorithms?

unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It can be compared to learning which takes place in the human brain while learning new things.

Unsupervised learning is a type of machine learning in which models are trained using unlabelled dataset and are allowed to act on that data without any supervision.



As we can see from the above figure, when the raw input is given to the machine learning models, it is able to group them according to certain hidden pattern and structure. Here the model can group fruits are apples, oranges and avocado based on their respective characteristics

- Unsupervised learning cannot be directly applied to a regression or classification problem because unlike supervised learning, we have the input data but no corresponding output data.
- Unsupervised learning, also known as unsupervised machine learning, uses machine learning algorithms to analyse and cluster unlabelled datasets.
- The goal of unsupervised learning is to find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format.



Suppose the unsupervised learning algorithm is given an input dataset containing images of different types of cats and dogs. The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset. The task of the unsupervised learning algorithm is to identify the image features on their own. Unsupervised learning algorithm will perform this task by clustering the image dataset into the groups according to similarities between images.

Why Unsupervised Learning?

There are multiple reasons for which unsupervised learning is important.

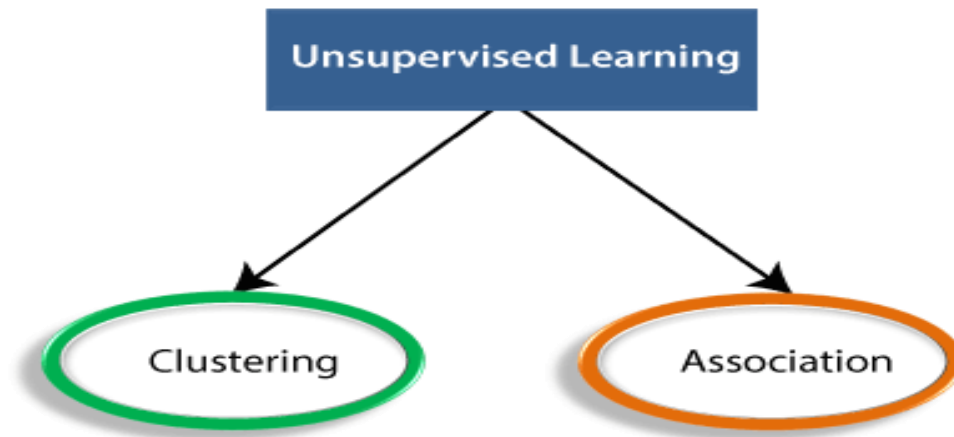
1. With human intervention, there are chances we might miss out on a certain patterns or criteria
2. Large datasets are very expensive, especially if everything to be labelled. Computers can mostly give unlabelled data so only few of them can be labelled manually.
3. With the help of clustering, it can find features that can help in the categorization of data.
4. It can help in scenarios where we do not know how many or what classes is the data divided.

Unsupervised classification

Unsupervised classified as:

Clustering: Clustering is a method of grouping the objects into clusters such that objects with most similarities.

Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.



Association: An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset.

Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis

Types of unsupervised algorithms

- K-means clustering.
- KNN (k-nearest neighbours)
- Hierarchical clustering.
- Anomaly detection.
- Neural Networks.
- Principle Component Analysis.
- Independent Component Analysis.
- Apriori algorithm.

Applications of Unsupervised learning algorithms.

Some major applications of unsupervised ML algorithms are:

- Clustering automatically divides the dataset into groups based on their similarities.
- Anomaly detection can discover unusual text or data points in your dataset. It is useful for finding fraudulent transactions.
- Association mining identifies sets of items that often occur together in your datapoints/dataset.
- Latent variable models are broadly used for data pre-processing. Like reducing the amount of features in a dataset or decomposing the dataset into multiple components.

Challenges associated with unsupervised algorithms

- Unsupervised learning is intrinsically more difficult than supervised learning as it does not have corresponding output.
- The result of the unsupervised learning algorithm might be less accurate as input data is not labelled, and algorithms do not know the exact output in advance

Applications of unsupervised algorithms

1. Products Segmentation
2. Customer Segmentation
3. Similarity Detection
4. Recommendation Systems
5. Labelling unlabelled datasets

1. Clustering

- Clustering is the process of grouping the given data into different clusters or groups. Unsupervised learning can be used to do clustering when we do not know exactly the information about the clusters.
- Elements in a group or cluster should be as similar as possible, and points in different groups should be as dissimilar as possible.
- Unsupervised learning can be used to do clustering when we do not know exactly the information about the clusters.
- It is used for analysing and grouping data, which does not include pre-labelled classes or class attributes. Clustering can be helpful for businesses to manage their data in a better way.

- For example, you can go to Walmart or a supermarket and see how different items are grouped and arranged there.
- Also, e-commerce websites like Amazon use clustering algorithms to implement a user-specific recommendation system.
- Here is another example. Let us say we have a YouTube channel. we may have a lot of data about the subscribers of our channel. If we want to detect groups of similar subscribers, then we may need to run a clustering algorithm. We do not need to tell the algorithm which group a subscriber belongs to. The algorithm can find those connections without our help. For example, it may tell you that 35% of your subscribers are from Canada, while 20% of them are from the United States.

These are some of the commonly used clustering algorithms:

1. Density-based.
2. Distribution-based.
3. Centroid-based.
4. Hierarchical-based.
5. K-means clustering algorithm.
6. DBSCAN clustering algorithm.
7. Gaussian Mixture Model algorithm.
8. BIRCH algorithm.

2. Visualization

- Visualization is the process of creating diagrams, images, graphs, charts, etc., to communicate some information. This method can be applied using unsupervised machine learning.
- For example, let us say you are a football coach, and you have some data about your team's performance in a tournament. You may want to find all the statistics about the matches quickly.
- You can feed the complex and unlabelled data to some visualization algorithm.
- These algorithms will output a two-dimensional or three-dimensional representation of your data that can easily be plotted. So, by seeing the plotted graphs, you can easily get a lot of information.
- This information will help you to maintain your winning formula, correct your previous mistakes, and win the ultimate trophy.

3. Dimensionality Reduction

- Dimensionality reduction is the process of reducing the number of random variables under consideration by getting a set of principal variables.
- In dimensionality reduction, the objective is to simplify the data without losing too much information. There can be a lot of similar information in your data.
- One method to do dimensionality reduction is to merge all those correlated features into one. This method is also called feature extraction.

These are some of the most common dimensionality reduction algorithms in machine learning:

✓ **Principal Component Analysis (PCA)**

✓ **Kernel PCA**

✓ **Locally-Linear Embedding**

4. Finding Association Rules

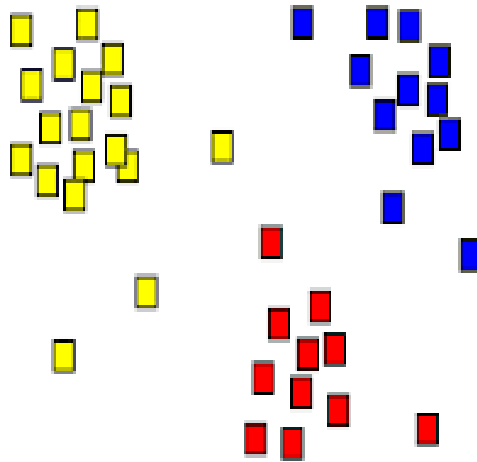
- This is the process of finding associations between different parameters in the available data. It discovers the probability of the co-occurrence of items in a collection, such as people that buy X also tend to buy Y.
- In association rule learning, the algorithm will deep dive into large amounts of data and find some interesting relationships between attributes.
- For example, when you go to Amazon and buy some items, they will show you products similar to those in advertisements, even when you are not on their website.
- This is a kind of association rule learning. Amazon can find associations between different products and customers. They know that if they show a particular advertisement to a particular customer, chances are high that he will buy the product.
- Thus, by using this method, they can increase their sales and revenue very highly. This leads to a more customized customer approach and is a pillar to customer satisfaction as well as retention.

5. Anomaly Detection

- Anomaly detection is the identification of rare items, events, or observations, which brings suspicions by differing significantly from the normal data.
- In this case, the system is trained with a lot of normal instances. So, when it sees an unusual instance, it can detect whether it is an anomaly or not.

- One important example of this is credit card fraud detection. You might have heard about a lot of events related to credit card fraud.
- This problem is now solved using anomaly detection techniques in machine learning. The system detects unusual credit card transactions to prevent fraud.

K-means Clustering



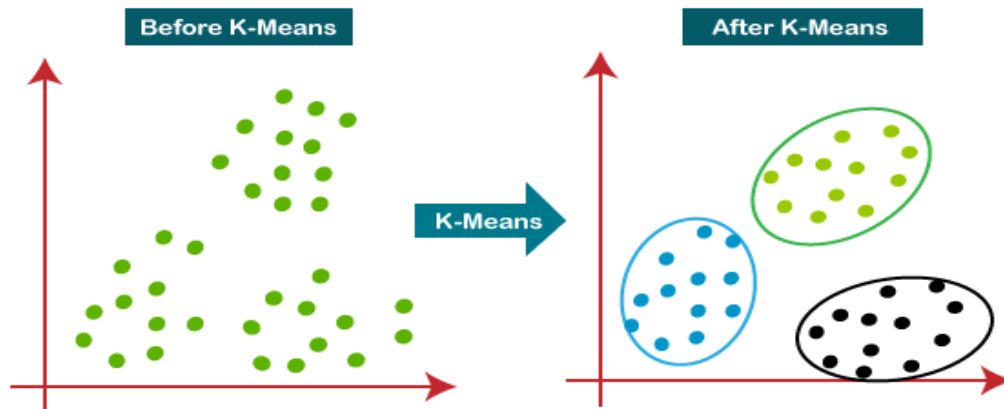
- ▶ K-Means Clustering is an unsupervised learning algorithm that is used to solve the clustering problems in machine learning or data science.
- ▶ K-Means Clustering is an Unsupervised Learning algorithm, which groups the unlabelled dataset into different clusters.
- ▶ Where K defines the number of pre-defined clusters that need to be created in the process, as if $K=2$, there will be two clusters, and for $K=3$, there will be three clusters, and so on.
- ▶ It is an iterative algorithm that divides the unlabelled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties.
- ▶ It allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabelled dataset on its own without the need for any training.
- ▶ It is a centroid-based algorithm, where each cluster is associated with a centroid. The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.
- ▶ The algorithm takes the unlabelled dataset as input, divides the dataset into k -number of clusters, and repeats the process until it does not find the best clusters. The value of k should be predetermined in this algorithm.

The k-means clustering algorithm mainly performs two tasks:

- Determines the best value for K center points or centroids by an iterative process.
- Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster.

Hence each cluster has datapoints with some commonalities, and it is away from other clusters.

The below diagram explains the working of the K-means Clustering Algorithm:



Working of k-means Clustering algorithm

Algorithm 1 *k*-means algorithm

- 1: Specify the number k of clusters to assign.
 - 2: Randomly initialize k centroids.
 - 3: **repeat**
 - 4: **expectation:** Assign each point to its closest centroid.
 - 5: **maximization:** Compute the new centroid (mean) of each cluster.
 - 6: **until** The centroid positions do not change.
-

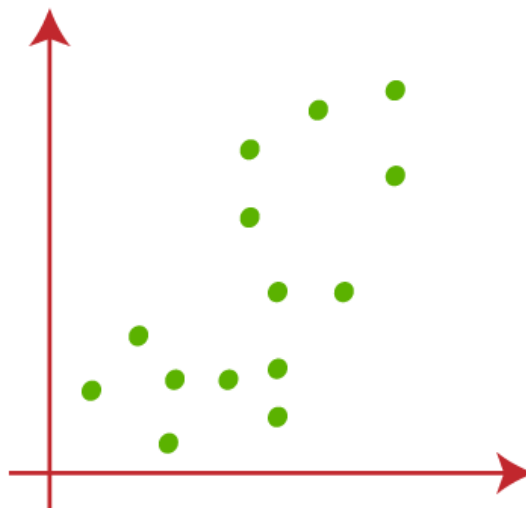
Working of k-means Clustering algorithm

Algorithm steps Of K Means

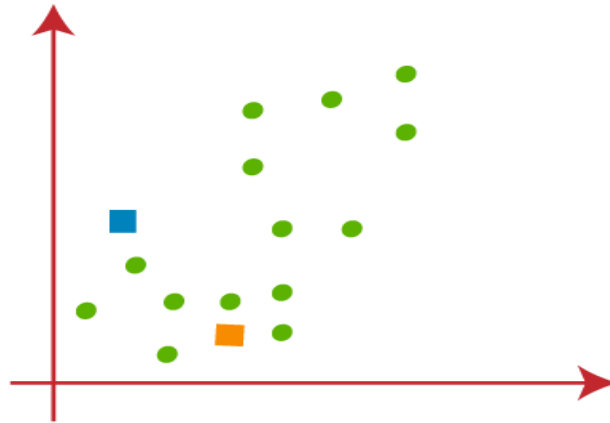
The working of the K-Means algorithm is explained in the below steps:

- ▶ Step-1: Select the value of K, to decide the number of clusters to be formed.
- ▶ Step-2: Select random K points which will act as centroids.
- ▶ Step-3: Assign each data point, based on their distance from the randomly selected points (Centroid), to the nearest/closest centroid which will form the predefined clusters.
- ▶ Step-4: place a new centroid of each cluster.
- ▶ Step-5: Repeat step no.3, which reassign each datapoint to the new closest centroid of each cluster.
- ▶ Step-6: If any reassignment occurs, then go to step-4 else go to Step 7.
- ▶ Step-7: FINISH

Suppose we have two variables M1 and M2. The x-y axis scatter plot of these two variables is given below:

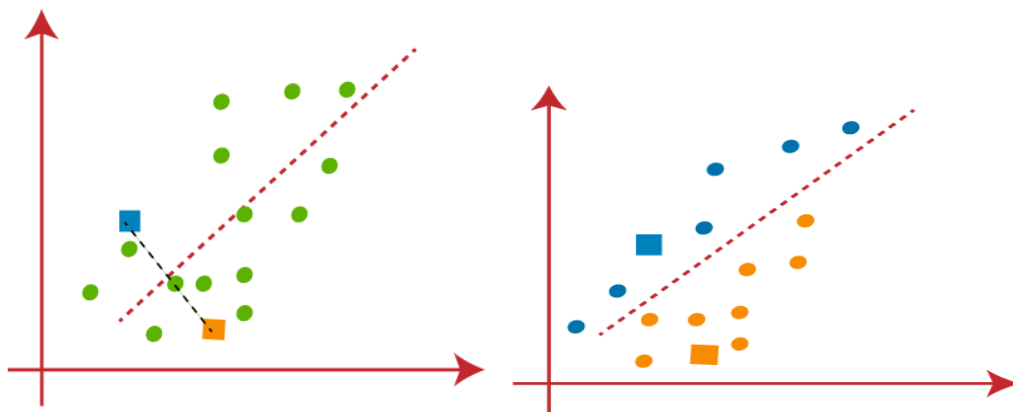


Let us take number k of clusters, i.e., $K=2$, to identify the dataset and to put them into different clusters. It means here we will try to group these datasets into two different clusters.

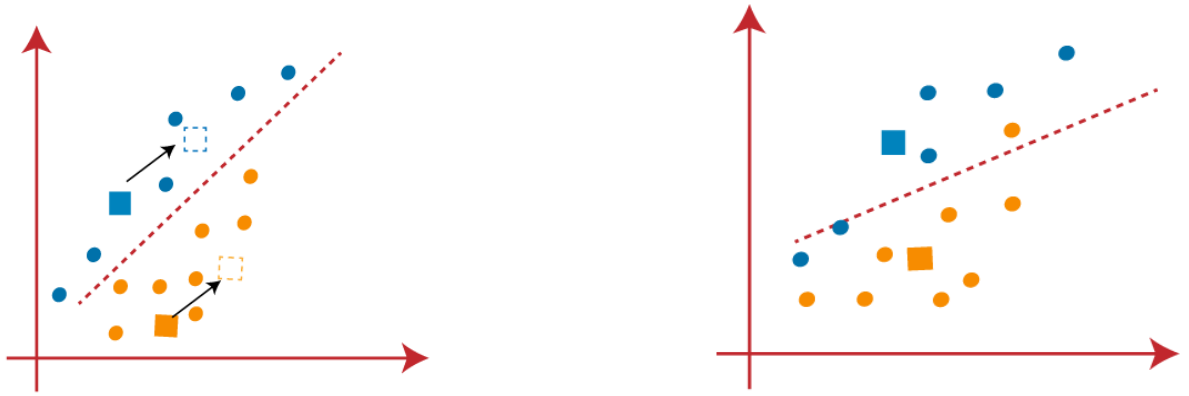


- We need to choose some random k points or centroid to form the cluster. These points can be either the points from the dataset or any other point.
- So, here we are selecting the below two points as k points, which are not the part of our dataset. Consider the below image.

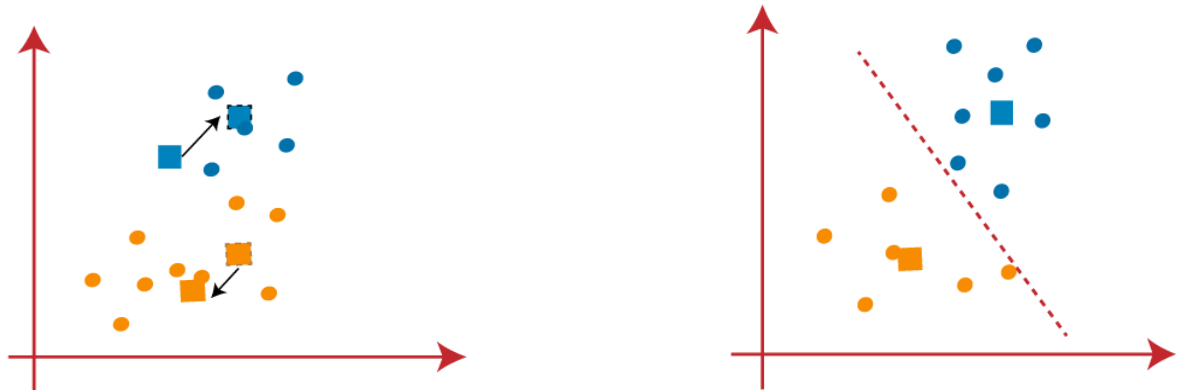
Now we will assign each data point of the scatter plot to its closest K -point or centroid. We will compute it by applying some mathematics that we have studied to calculate the distance between two points. So, we will draw a median between both the centroids. Consider the below image:



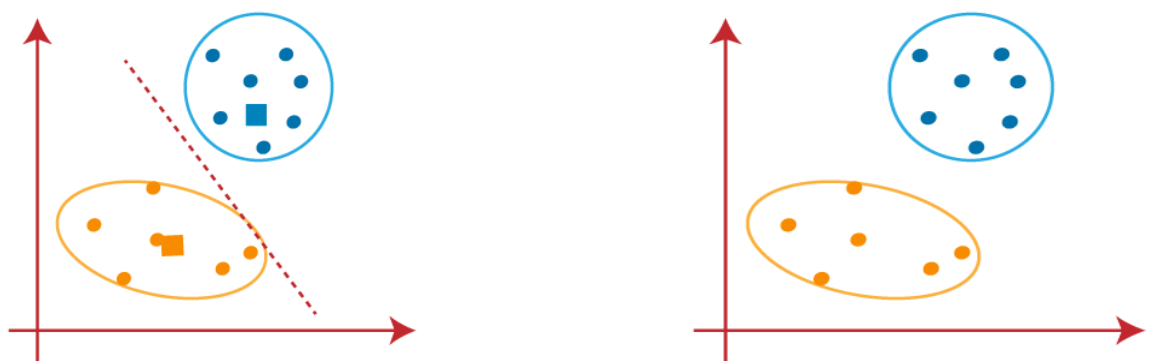
As we need to find the closest cluster, so we will repeat the process by choosing a new centroid. To choose the new centroids, we will compute the centre of gravity of these centroids, and will find new centroids as below:



We will repeat the process by finding the center of gravity of centroids, so the new centroids will be as shown in the below image:



We can see in the above image; there are no dissimilar data points on either side of the line, which means our model is formed. Consider the below image:



How to choose the value of "K number of clusters" in K-means Clustering?

- The performance of the K-means clustering algorithm depends upon highly efficient clusters that it forms. But choosing the optimal number of clusters is a big task.

Elbow Method

- The Elbow method is one of the most popular ways to find the optimal number of clusters. This method uses the concept of WCSS value. **WCSS** stands for **Within Cluster Sum of Squares**, which defines the total variations within a cluster. The formula to calculate the value of WCSS (for 3 clusters) is given below:

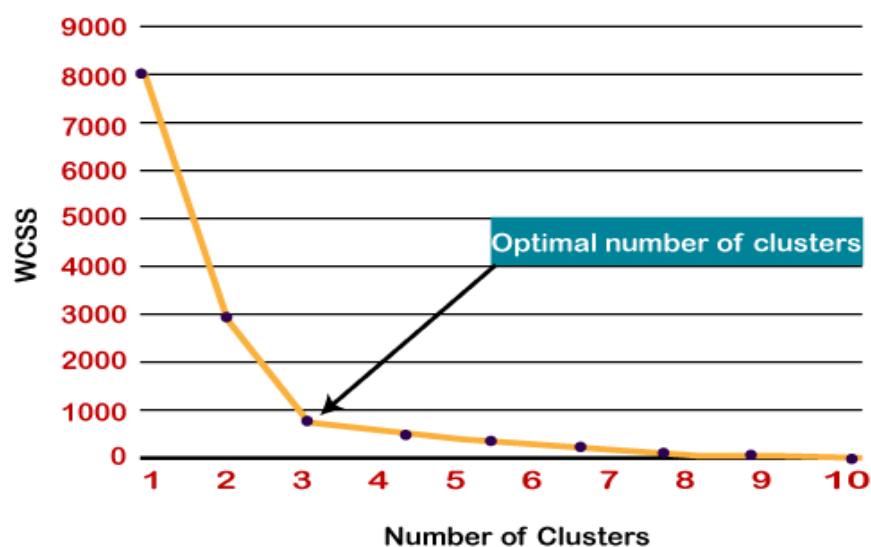
$$WCSS = \sum_{P_i \text{ in Cluster1}} \text{distance}(P_i C_1)^2 + \sum_{P_i \text{ in Cluster2}} \text{distance}(P_i C_2)^2 + \sum_{P_i \text{ in Cluster3}} \text{distance}(P_i C_3)^2$$

$\sum_{P_i \text{ in Cluster1}} \text{distance}(P_i C_1)^2$: It is the sum of the square of the distances between each data point and its centroid within a cluster1 and the same for the other two terms.

To measure the distance between data points and centroid, we can use any method such as Euclidean distance or Manhattan distance.

To find the optimal value of clusters, the elbow method follows the below steps:

- It executes the K-means clustering on a given dataset for different K values (ranges from 1-10).
- For each value of K, calculates the WCSS value.
- Plots a curve between calculated WCSS values and the number of clusters K.
- The sharp point of bend or a point of the plot looks like an arm, then that point is considered as the best value of K.



Program to demonstrate K-means unsupervised algorithm (mall customer dataset is used)

```
# importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd

dataset = pd.read_csv('Mall_Customers_data.csv') (income v/s spending column)
x = dataset.iloc[:, [3, 4]].values

#finding optimal number of clusters using the elbow method
from sklearn.cluster import KMeans
wcss_list= []

#Using for loop for iterations from 1 to 10.
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state= 42)
    kmeans.fit(x)
    wcss_list.append(kmeans.inertia_)

mtp.plot(range(1, 11), wcss_list)
mtp.title('The Elbow Method Graph')
mtp.xlabel('Number of clusters(k)')
mtp.ylabel('wcss_list')
mtp.show()

#training the K-means model on a dataset
kmeans = KMeans(n_clusters=5, init='k-means++', random_state= 42)
y_predict= kmeans.fit_predict(x)

#visualizing the clusters
mtp.scatter(x[y_predict == 0, 0], x[y_predict == 0, 1], s= 100, c= 'blue', label= 'Cluster 1') #for first cluster

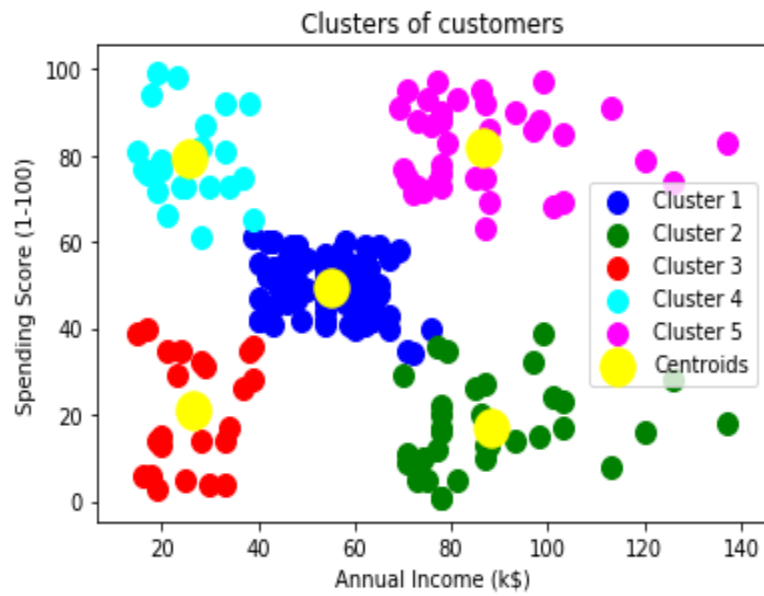
mtp.scatter(x[y_predict == 1, 0], x[y_predict == 1, 1], s= 100, c= 'green', label= 'Cluster 2') #for second cluster

mtp.scatter(x[y_predict== 2, 0], x[y_predict == 2, 1], s= 100, c= 'red', label= 'Cluster 3') #for third cluster

mtp.scatter(x[y_predict == 3, 0], x[y_predict == 3, 1], s= 100, c= 'cyan', label= 'Cluster 4') #for fourth cluster

mtp.scatter(x[y_predict == 4, 0], x[y_predict == 4, 1], s= 100, c= 'magenta', label= 'Cluster 5') #for fifth cluster
```

```
mtp.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1], s = 300, c = 'yellow', label = 'Centroid')
mtp.title('Clusters of customers')
mtp.xlabel('Annual Income (k$)')
mtp.ylabel('Spending Score (1-100)')
mtp.legend()
mtp.show()
```



The output image is clearly showing the five different clusters with different colours. The clusters are formed between two parameters of the dataset; Annual income of customer and Spending. We can change the colours and labels as per the requirement or choice. We can also observe some points from the above patterns, which are given below:

- **Cluster1** shows the customers with average salary and average spending so we can categorize these customers as
- Cluster2 shows the customer has a high income but low spending, so we can categorize them as **careful**.
- Cluster3 shows the low income and low spending so they can be categorized as sensible.
- Cluster4 shows the customers with low income with very high spending so they can be categorized as **careless**.
- Cluster5 shows the customers with high income and high spending so they can be categorized as target, and these customers can be the most profitable customers for the mall owner.

Program to demonstrate k-means clustering using iris dataset:

```
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
```

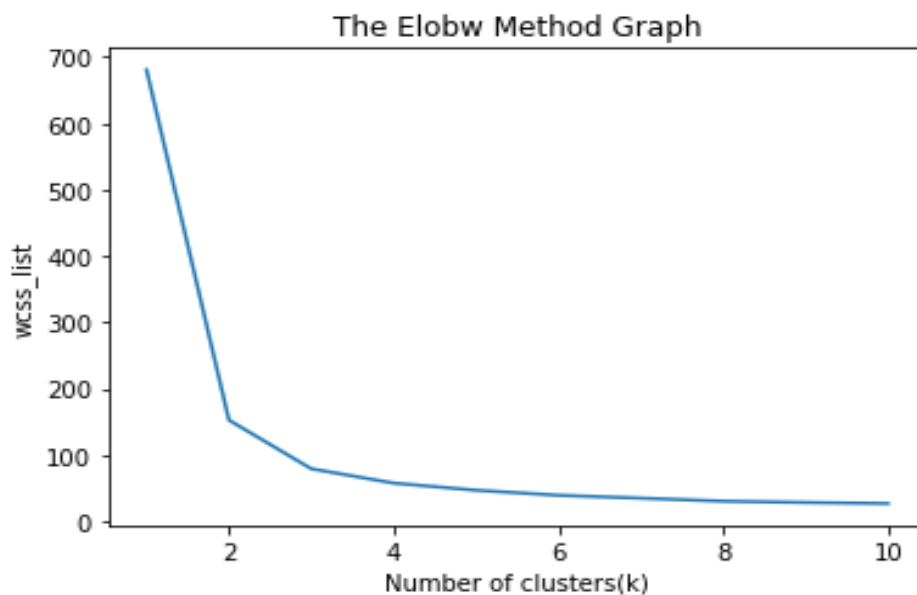
In [17]:

```
df = pd.read_csv("C:/Users/Shilpa/Desktop/dataset/Iris.csv")
x = df.iloc[:,1:5].values
print(x)
```

```
from sklearn.cluster import KMeans
wcss_list= []
```

```
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state= 42)
    kmeans.fit(x)
    wcss_list.append(kmeans.inertia_)
```

```
mtp.plot(range(1,11), wcss_list)
mtp.title('The Elbow Method Graph')
mtp.xlabel('Number of clusters(k)')
mtp.ylabel('wcss_list')
mtp.show()
```



```
kmeans = KMeans(n_clusters=3, init='k-means++', random_state= 42)
y_predict= kmeans.fit_predict(x)
```

```
mtp.scatter(x[y_predict == 0, 0], x[y_predict == 0, 1], s = 100, c = 'blue', label = 'Iris-setosa') #for first cluster
```

```
mtp.scatter(x[y_predict == 1, 0], x[y_predict == 1, 1], s = 100, c = 'green', label = 'Iris-versicolor') #for second cluster
```

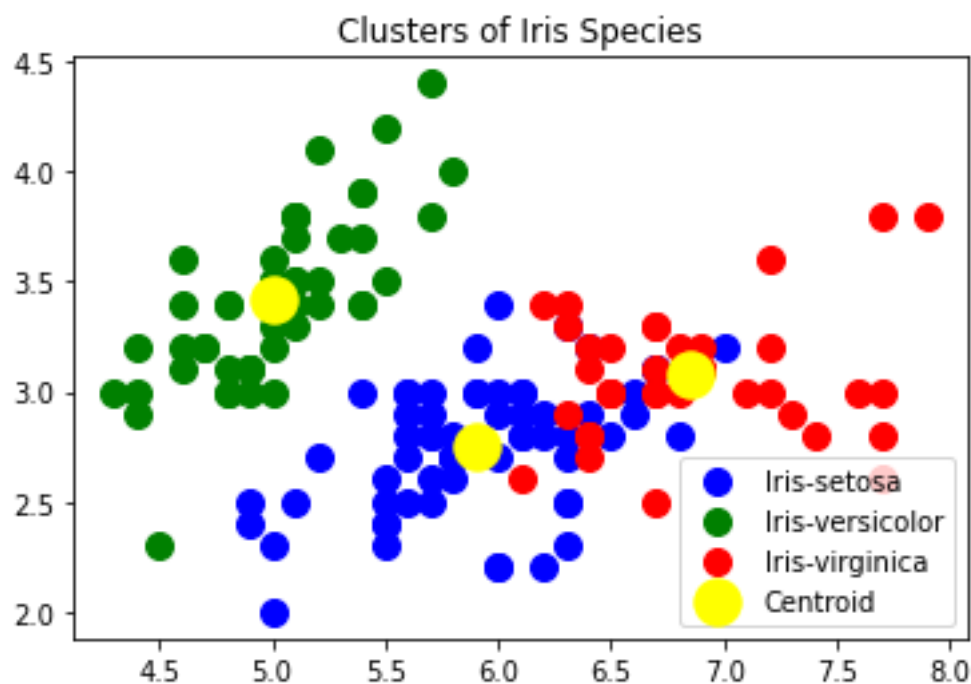
```
mtp.scatter(x[y_predict == 2, 0], x[y_predict == 2, 1], s = 100, c = 'red', label = 'Iris-virginica') #for third cluster
```

```
mtp.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s = 300, c = 'yellow', label = 'Centroid')
```

```
mtp.title('Clusters of Iris Species')
```

```
mtp.legend()
```

```
mtp.show()
```



As we can see from the above figure the iris dataset can be classified into three categories of **iris-setosa**, **iris-versicolor** and **iris-virginica**.

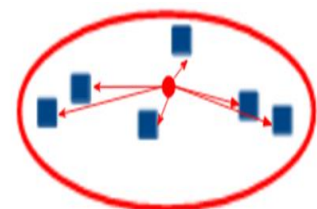
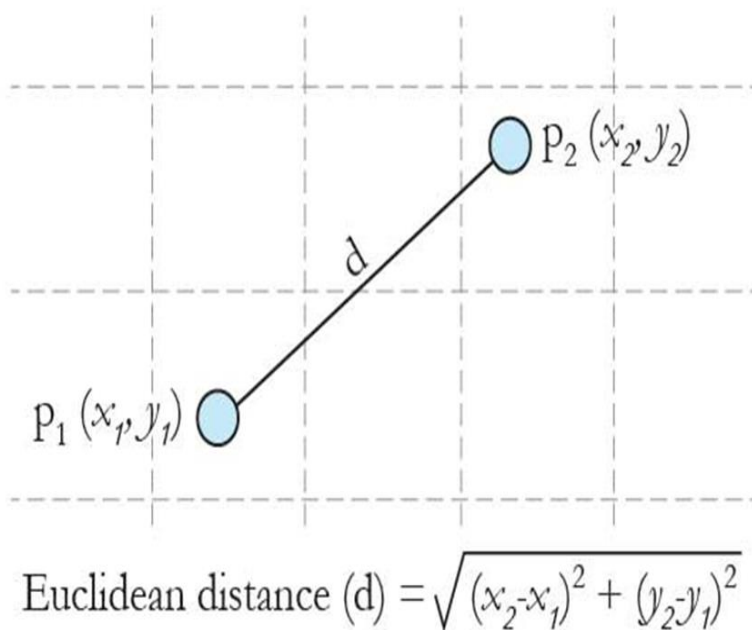
Evaluation metrics of unsupervised learning algorithm

1. Inertia

- inertia **measures how well a dataset was clustered by K-Means**. It is calculated by measuring the distance between each data point and its centroid, squaring this distance, and summing these squares across one cluster. A good model is one with low inertia AND a low number of clusters (K).

$$\sum_{i=1}^N (x_i - C_k)^2$$

- N is the number of samples within the data set, C is the center of a cluster. So the Inertia simply computes the squared distance of each sample in a cluster to its cluster center and sums them up.
- This process is done for each cluster and all samples within that data set. The smaller the Inertia value, the more coherent are the different clusters. When as many clusters are added as there are samples in the data set, then the Inertia value would be zero



Intra cluster distance

A point is represented using x and y co-ordinates, hence the Euclidean distance between the two points can be calculated using above formula

A sample example showing how to assign data points to cluster.**Assigning Clusters to data points**

Centroid 1 = (4,3)
 Centroid 2 = (11,6)
 Centroid 3 = (8,10)

Datapoint = (7,4)

Apply Euclidean distance formula : $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

We need to find which centroid has minimum distance with the given datapoint.

With centroid 1 : $((7 - 4)^2 + (4 - 3)^2) = 10$

With centroid 2 : $((7 - 11)^2 + (4 - 6)^2) = 20$

With centroid 3 : $((7 - 8)^2 + (4 - 10)^2) = 37$

As we can compare, datapoint (7,4) is closest to centroid 1, therefore it will be assigned in cluster 1

Similarly, Euclidean distance is calculated for every data point. After the formation of clusters, the next step is to update or recompute the centroid values. This is done by taking the Euclidean average or mean value of the data points in a particular cluster.

Recomputing centroid of the cluster

Suppose we have 3 datapoints in this cluster

Data point 1 = (7,4)
 Data point 2 = (5,2)
 Data point 3 = (3,2)

Old centroid : (4,3)

Updated centroid = average of datapoints in this cluster

Updated centroid : $C_x = (7+5+3)/3 = 5$, $C_y = (4+2+2)/3 = 2.6$

Updated centroid of this cluster is (5,2.6)

Stop criteria

1. We can stop our training when even after many iterations our centroids are stable, that is they are the same and fixed.
2. We can stop our training when even after many iterations our data points remain in the same cluster. That they do not change their cluster anymore.
3. We can stop our training when the fixed or maximum number of iterations is reached. This is because an insufficient number of iterations might give us poor results and hence unstable clusters.

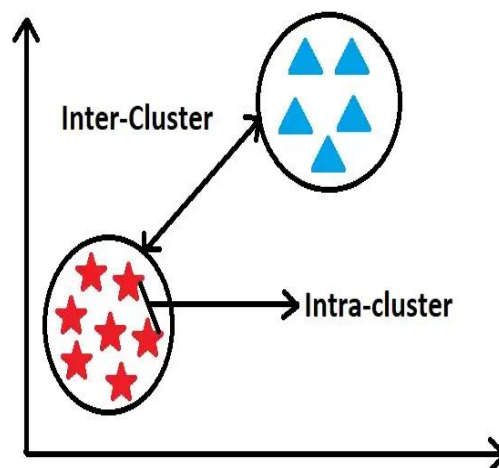
2. DUNN INDEX

The Dunn index **aims to identify dense and well-separated clusters**. It is defined as the ratio between the minimal inter-cluster distance to maximal intra-cluster distance.

Intra-cluster: The distance between two similar data points belonging to the same cluster.

Inter-cluster: The distance between two dissimilar data points belonging to different clusters.

The main objective of any good clustering algorithm is to reduce the intra-cluster distance and maximize the inter-cluster distance. One of the main performance metrics that is used for clustering is the Dunn Index parameter.



The Dunn Index (DI) is one of the clustering algorithms evaluation measures. It is most commonly used to evaluate the goodness of split by a K-Means clustering algorithm for a given number of clusters.

Clusters are far apart

$$\text{Dunn Index} = \frac{\min(\text{Inter cluster distance})}{\max(\text{Intra cluster distance})}$$

min(Inter cluster distance)

$$\text{Dunn Index} = \frac{\min(\text{Inter cluster distance})}{\max(\text{Intra cluster distance})}$$

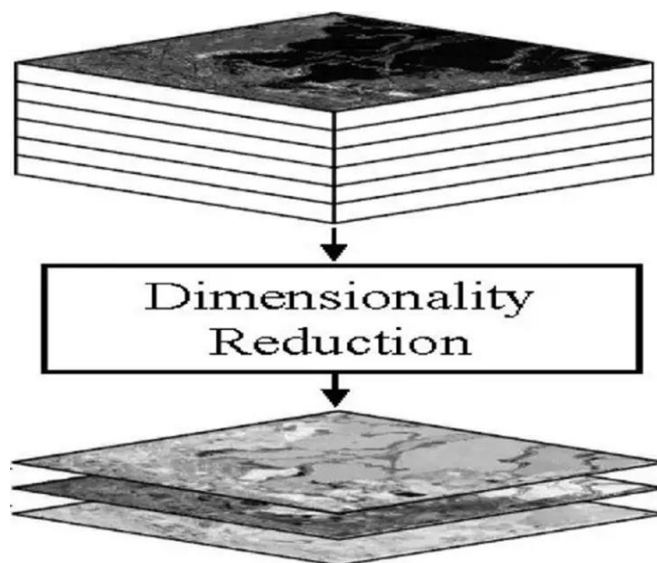
max(Intra cluster distance)

Clusters are compact

$$D = \frac{\min_{1 \leq i < j \leq n} d(i, j)}{\max_{1 \leq k \leq n} d'(k)}$$

Dimensionality reduction

- Dimensionality reduction simply refers to **the process of reducing the number of attributes in a dataset while keeping as much of the variation in the original dataset as possible**. It is a data preprocessing step meaning that we perform dimensionality reduction before training the model.



Problem with high dimensional data?

- It can mean high computational cost to perform learning.
- It often leads to over-fitting when learning a model, which means that the model will perform well on the training data but poorly on test data.
- Data are rarely randomly distributed in high-dimensions and are highly correlated, often with spurious correlations.
- The distances between a nearest and farthest data point can become equidistant in high dimensions, that can hamper the accuracy of some distance-based analysis tools.

Why do we need Dimensionality Reduction?

- ▶ Dimensionality reduction helps with these problems, while trying to preserve most of the relevant information in the data needed to learn accurate, predictive models.
- ▶ There are often too many factors based on which the final prediction is done. These factors are basically variables called features.
- ▶ The higher the number of features, the harder it gets to visualize the training set and then work on it.
- ▶ Sometimes, most of these features are correlated, and hence redundant. This is where dimensionality reduction algorithms come into play.

Importance of Dimensionality reduction

- ▶ It reduces the time and storage space required.
- ▶ It helps Remove multi-collinearity which improves the interpretation of the parameters of the machine learning model.
- ▶ It becomes easier to visualize the data when reduced to very low dimensions such as 2D or 3D.
- ▶ It avoids the curse of dimensionality.
- ▶ It removes irrelevant features from the data, because having irrelevant features in the data can decrease the accuracy of the models and make your model learn based on irrelevant features.
- ▶ Visualization of high-dimensional data can be achieved through dimensionality reduction
- ▶ Dimensionality reduction can be applied to mitigate the problem of overfitting
- ▶ Dimensionality reduction saves a lot of computational resources when training models
- ▶ Dimensionality reduction automatically removes multicollinearity.

Not all variables in your data are independent. Some input variables may correlate with the other input variables in the dataset. This is referred to as *multicollinearity* which can negatively affect the performance of your regression and classification models.

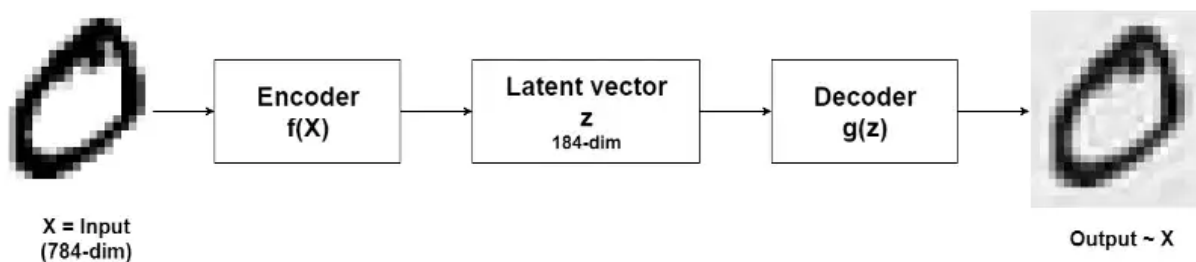
- ▶ Dimensionality reduction can be used for image compression

Dimensionality reduction reduces the size of your dataset while keeping much of the variability in the original data as possible. A similar kind of approach can be used for image

compression. So, in image compression, we reduce the number of pixels of an image while keeping as much of the quality in the original image as possible.

Dimensionality reduction improves the accuracy of models Dimensionality reduction can be used to compress neural network architectures

- ▶ This can be achieved by using a special neural network architecture called *Autoencoders* that compresses high-dimensional input data into a lower dimension. In other words, it finds a compressed representation of the input data.
- ▶ An autoencoder has two functions that are also known as operators:
 - Encoder: This is a non-linear function that transforms the input data into a lower-dimensional form called the *latent vector*.
 - Decoder: This is also a non-linear function that takes the latent vector as the input and constructs another output that is much similar to the original input. The goal is to minimize the *reconstruction error*.



- ▶ The encoder takes the X (784-dim) as the input and transforms it into a lower-dimensional latent vector (184-dim) which is taken by the decoder as its input. The output of the decoder is much similar to the original input, X but not exactly the same. The dissimilarity between the input and output is measured by the *reconstruction error* or the *loss function* that should be kept as minimum as possible.

Common methods to perform Dimension Reduction

Dimensionality Reduction Techniques

- Feature selection. ...
- Feature extraction. ...
- Principal Component Analysis (PCA) ...
- Non-negative matrix factorization (NMF) ...
- Linear discriminant analysis (LDA) ...
- Generalized discriminant analysis (GDA) ...
- Missing Values Ratio. ...
- Low Variance Filter.

Dimensionality reduction using PCA technique

Principal Component Analysis (PCA)

- Principal component analysis, or PCA, is a dimensionality-reduction method that is often used to reduce the dimensionality of large data sets, by **transforming a large set of variables into a smaller one that still contains most of the information in the large set.**
- **the main purpose of principal component analysis (pca) simplifies the complexity in high-dimensional data while retaining trends and patterns.**
- PCA is mainly used as the dimensionality reduction technique in various AI applications such as **computer vision, image compression**, etc. It can also be used for finding hidden patterns if data has high dimensions. Some fields where PCA is used are finance, data mining etc.
- PCA can help us improve performance at a very low cost of model accuracy. Other benefits of PCA include reduction of noise in the data, feature selection (to a certain extent), and the ability to produce independent, uncorrelated features of the data.
- PCA improves the performance of the ML algorithm as it eliminates correlated variables that don't contribute in any decision making. PCA helps in overcoming data overfitting issues by decreasing the number of features. PCA results in high variance and thus improves visualization

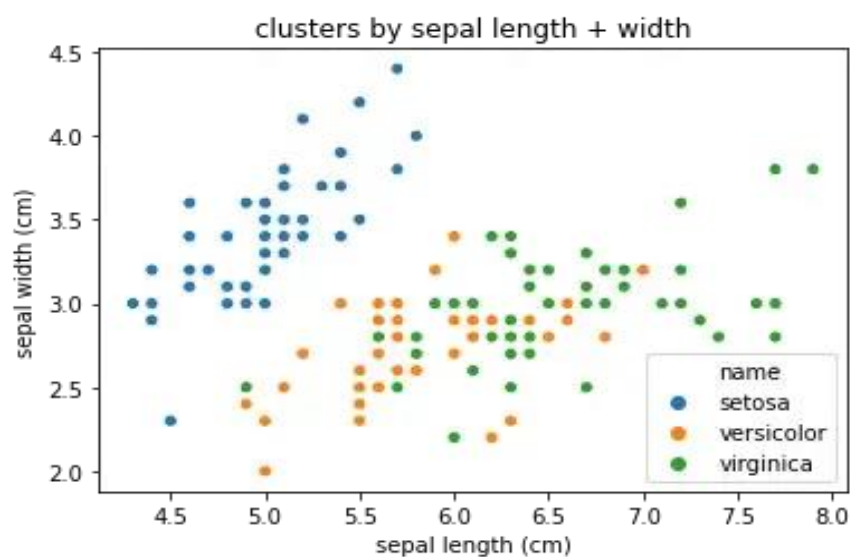
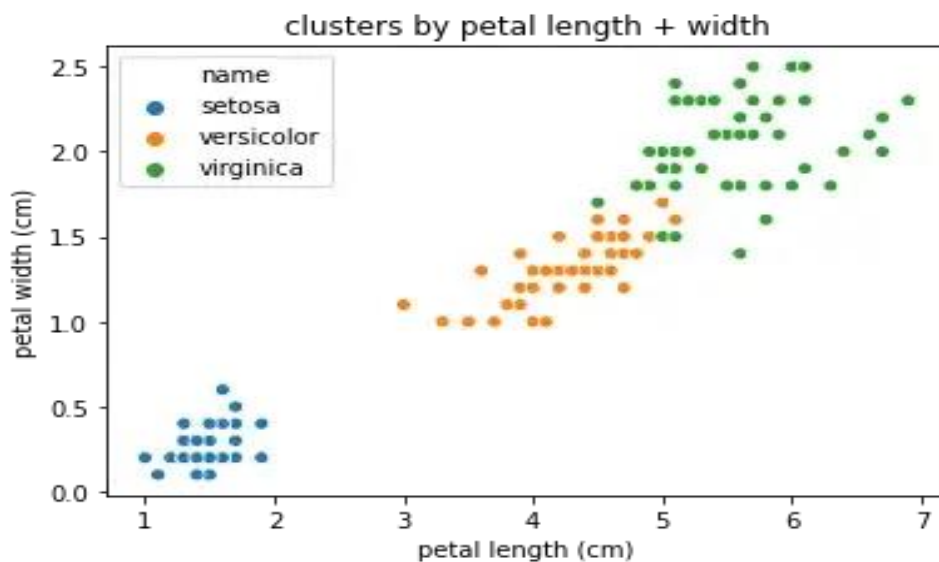
Uses of PCA

- PCA is used to visualize multidimensional data.
- It is used to reduce the number of dimensions in healthcare data.
- PCA can help resize an image.
- It can be used in finance to analyse stock data and forecast returns.
- PCA helps to find patterns in the high-dimensional datasets.

PCA ANALYSIS AND PROGRAM FOR IRIS DATASET

As we can see iris dataset has the four features sepal length ,sepal width ,petal length, petal width, these three specifications helps in classification the iris category it belongs to.

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2



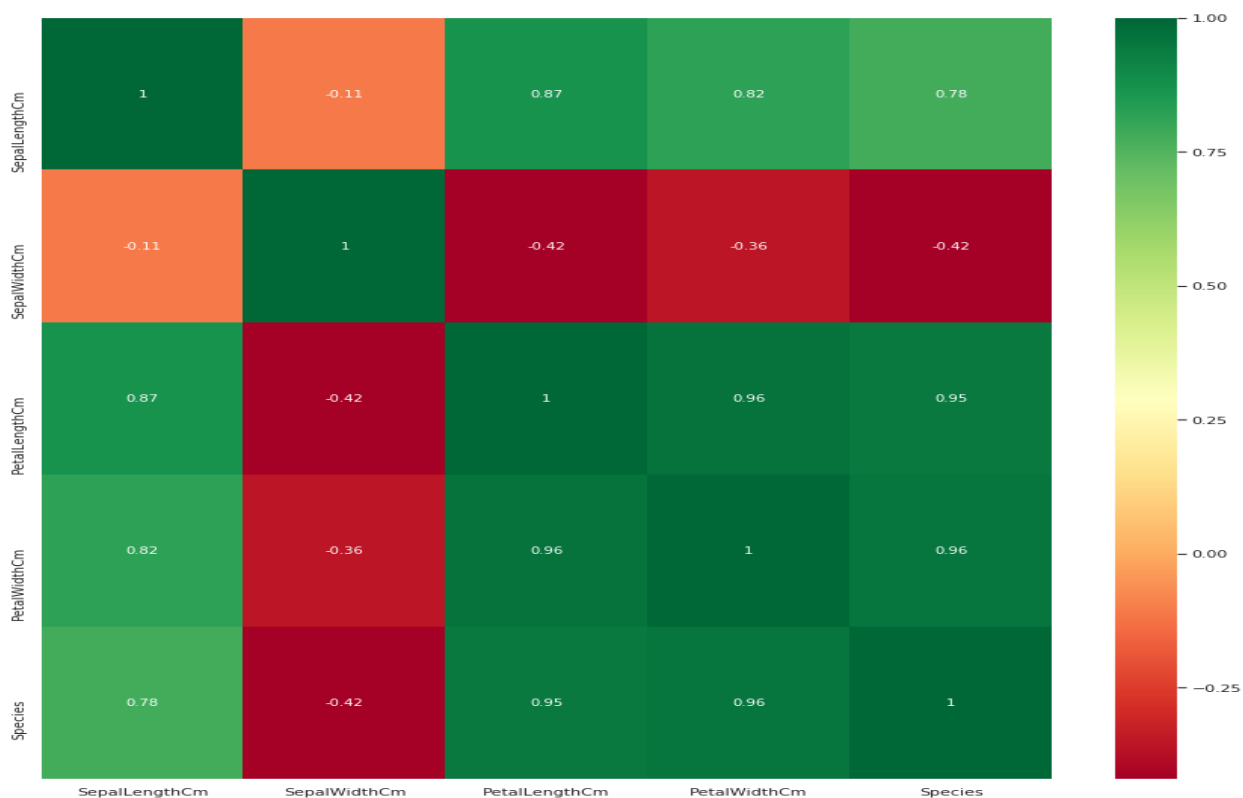
When scatter plot is drawn against (petal length and petal width) & (sepal length and sepal width) In both cases we can easily distinguish the three types of Iris. This is important, because it tells us that both the sepal features and the petal feature independently contain explanatory information relating to the type of Iris, i.e., the target class.

If we could only choose petals or sepals to attempt classifying, then of course the petal features would yield much better results, but not perfect:



The *Iris virginica* and *Iris versicolor* clusters blend a bit, and are not linearly separable. In some cases, the data may even be linearly separable after applying a transformation to the data, but no such pattern seems to exist here.

Thus, we apply PCA! Even though the sepal features seem even worse for classifying the target (i.e. they are even less linearly separable by Iris type), they nevertheless contain important *information*, or statistical variance, which may reveal more linear separability in different dimensions while also reducing the amount of features! **This is why PCA is useful.**



- As always, there is a slight amount of preparation required before applying PCA.
- Because the features in the Iris dataset are on totally different scales (e.g. the sepal lengths are much longer than the petal widths), we need to scale them so that the new principal components treat all features equally via singular value decomposition.

```
from sklearn.preprocessing import StandardScaler
features = df[['p_length', 'p_width', 's_length', 's_width']]
scaled_features = StandardScaler().fit_transform(features)
```

Program to demonstrate dimensionality reduction using principal component analysis (PCA) for iris dataset.

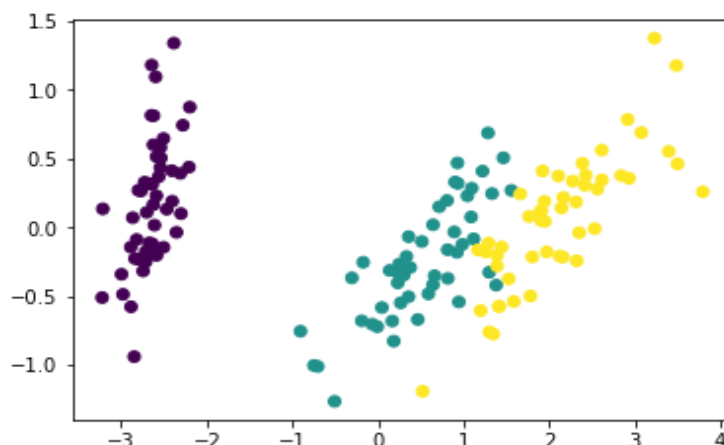
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.decomposition import PCA
```

```
iris=datasets.load_iris()
x=iris.data
y=iris.target
```

```
print(x.shape)
print(y.shape)
```

```
pca=PCA(n_components=2)
pca.fit(x)
print(pca.components_)
```

```
x=pca.transform(x)
print(x.shape)
plt.scatter(x[:,0],x[:,1],c=y)
```



```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

x_train, x_test,y_train,y_test=train_test_split(x,y,test_size=0.2)
res=DecisionTreeClassifier()
res.fit(x_train,y_train)

y_predict=res.predict(x_test)
print(accuracy_score(y_test,y_predict))
```

MLOP's

- *A Machine Learning pipeline is a process of automating the workflow of a complete machine learning task.*
- It can be done by enabling a sequence of data to be transformed and correlated together in a model that can be analysed to get the output.
- A typical pipeline includes raw data input, features, outputs, model parameters, ML models, and Predictions.
- Moreover, an ML Pipeline contains multiple sequential steps that perform everything ranging from data extraction and pre-processing to model training and deployment in Machine learning in a modular approach.
- It means that *in the pipeline, each step is designed as an independent module, and all these modules are tied together to get the final result.*

What is MLOP's?

- ▶ MLOP's stands for **Machine Learning Operations** for production.
- ▶ MLOP's is a core function of Machine Learning engineering, focused on streamlining the process of taking machine learning models to production, and then maintaining and monitoring them.
- ▶ MLOP's Engineers build and maintain a platform to enable the development and deployment of machine learning models.
- ▶ MLOps, also known as Machine Learning Operations for Production, is a set of standardized practices that can be utilized to build, deploy, and govern the lifecycle of [ML models](#).
- ▶ This setup helps to ease the interaction among cross-functional teams and provides an automated platform to keep track of everything required for the complete cycle of ML models.

- MLOps practices also result in increased scalability, security, and reliability of the ML systems, leading to shorter development cycles and escalated profits from the [ML projects](#).

How do you make a pipeline for the machine learning model?

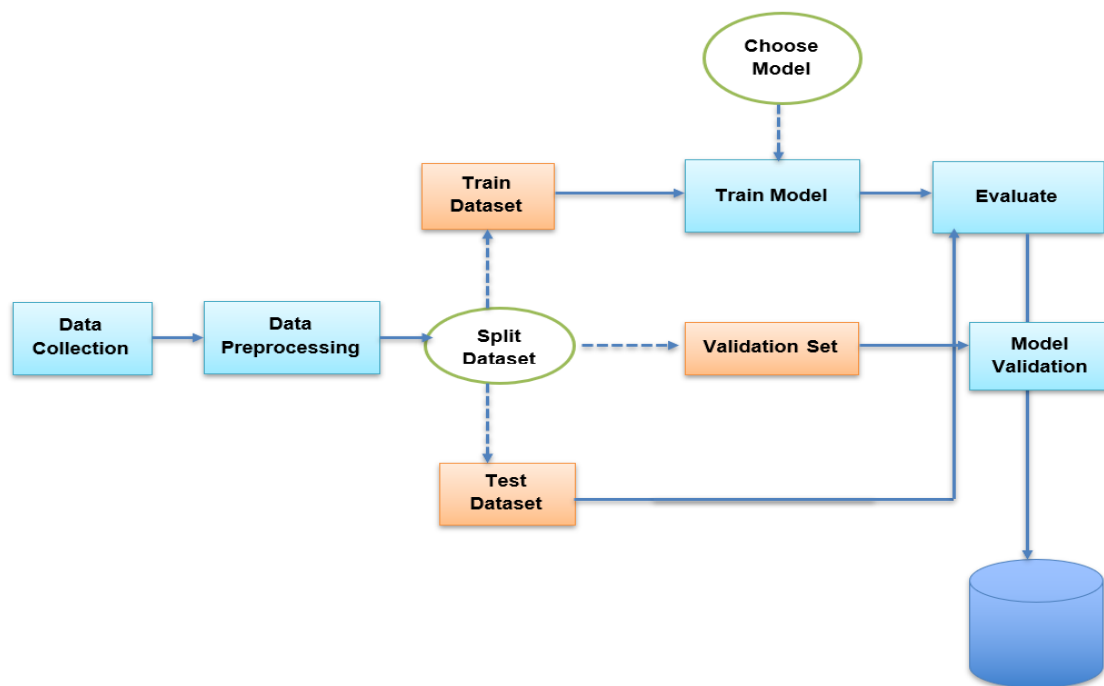
- There are various stages in a machine learning pipeline architecture, mainly- Data pre-processing, Model training, Model evaluation, and Model deployment.
- Each stage of the data pipeline passes processed data to the next step, i.e., it gives the output of one phase as input data into the next phase

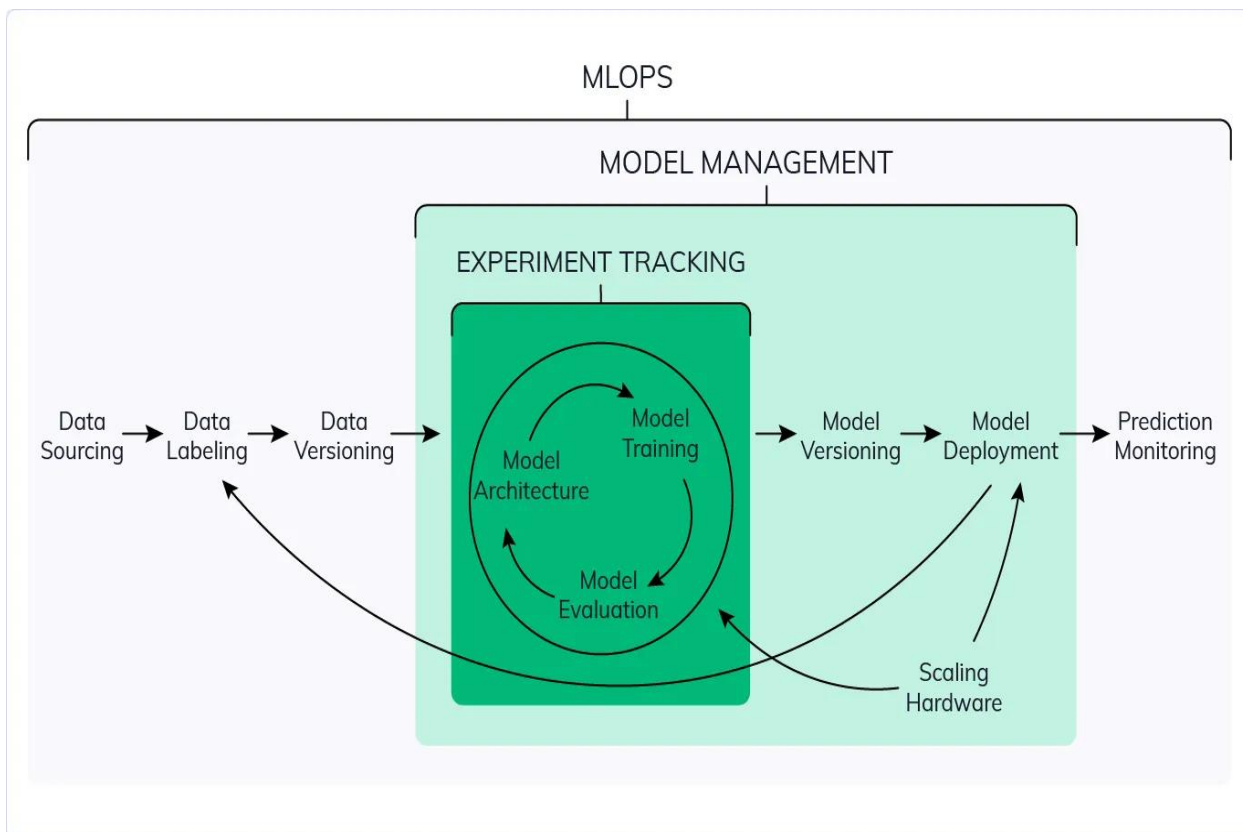
Why MLOps?

There are many goals enterprises want to achieve through MLOps. Some of the common ones are:

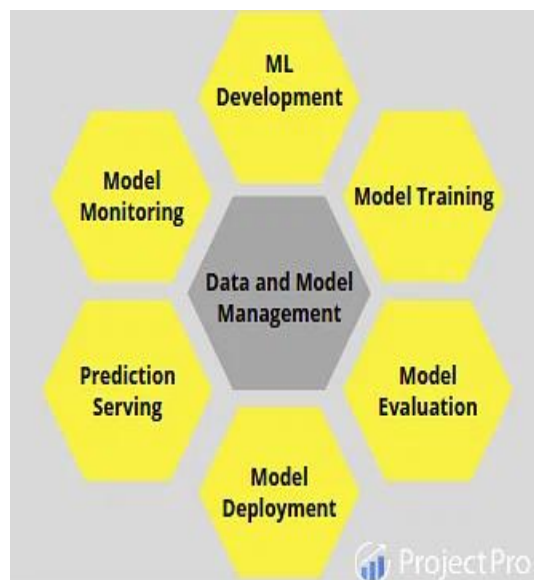
- Automation
- Scalability
- Reproducibility
- Monitoring
- Governance

Machine learning Workflow





MLOps Lifecycle



1. **ML Development:** This is the basic step that involves creating a complete pipeline beginning from data processing to model training and evaluation codes.
2. **Model Training:** Once the setup is ready, the next logical step is to train the model. Here, continuous training functionality is also needed to adapt to new data or address specific changes.

3. **Model Evaluation:** Performing inference over the trained model and checking the accuracy/correctness of the output results.
4. **Model Deployment:** When the proof of concept stage is accomplished, the other part is to deploy the model according to the industry requirements to face the real-life data.
5. **Prediction Serving:** After deployment, the model is now ready to serve predictions over the incoming data.
6. **Model Monitoring:** Over time, problems such as concept drift can make the results inaccurate hence continuous monitoring of the model is essential to ensure proper functioning.

Data and Model Management: It is a part of the central system that manages the data and models. It includes maintaining storage, keeping track of different versions, ease of accessibility, security, and configuration across various cross-functional teams.

Versioning of Machine learning models

- ▶ A developer is accountable for questions such as the dataset used to train the model; hyperparameters; pipeline used to create the model; last deployed version of the model etc.
- ▶ This calls for the application of version control in machine learning models. The accuracy of the dataset varies when you update and tinker with different parts of the model. With versioning, developers can scope out the best model and its tradeoffs.
- ▶ A machine learning model can fall flat for several reasons. For example, while adding more data or incorporating performance improvement measures. In case of such failures, version modelling helps in quickly reverting to the previous working version.
- ▶ Machine learning models can be very complex. Factors such as datasets, training and testing, frameworks, among others, account for a model's success. Version control helps in keeping dependency tracking.
- ▶ Major updates to machine learning models are not usually rolled out at once. To ensure better performance and failure tolerance, the ML models are released in phases. Versioning allows the deployment of the right versions at the right time.
- ▶ Model versioning is an essential component of AI/ML governance for organisations to control access, implement policy, and track model activity.
- ▶ **Git:** Git is the standard versioning protocol used across the board to monitor and version control software development and deployment. Git tracks changes made to the code and help in implementing, storing, and merging changes.
- ▶ Git also comes with a few drawbacks. It is a challenge to keep all the folders in sync in Git. The model checkpoints and data size occupy the bulk of the space. Many users alternatively store the datasets in cloud servers such as Amazon 3, reproducible codes in Git, and generate

models on the fly. But working with multiple data sets breeds confusion. Further, improper documentation of data changes and upgrades can result in the model losing the context.

- ▶ **DVC:** Data Version Control is a Git extension. It is a streamlined version of combining Git with ML specific functionality for data management. DVC can run top of any Git repository and is compatible with the Git server or provider. DVC also offers all the advantages of the distributed version control system, such as lock-free, local branching, and versioning.
- ▶ **Pachyderm:** It delivers robust data versioning and data lineage to the machine learning loop. It also provides a flexible pipeline system that can use any tool or framework in the transformation steps. Pachyderm uses containers to execute different pipeline steps and solves data provenance issues by tracking data commits and optimizing the pipeline.

Machine learning metadata (MLMD): It is a recently introduced library from the Tensorflow team to track the entire ML workflow's full lineage. The complete lineage includes steps such as data ingestion, preprocessing, validation, training, and deployment. MLMD can be used to trace bad models back to the datasets.

Machine learning model registry

- ▶ A model registry is a repository used to store and version trained machine learning (ML) models. Model registries greatly simplify the task of tracking models as they move through the ML lifecycle, from training to production deployments and ultimately retirement.
- ▶ MLflow Model Registry is a centralized model repository and a UI and set of APIs that enable you to manage the full lifecycle of MLflow Models. Model Registry provides: Chronological model lineage (which MLflow experiment and run produced the model at a given time).

MLflow is an MLOps tool that enables data scientist to quickly productionize their Machine Learning projects. To achieve this, MLFlow has four major components which are Tracking, Projects, Models, and Registry. MLflow lets you train, reuse, and deploy models with any library and package them into reproducible steps.

